Probing Functional Correctness in Diffusion Language Models

Anonymous ACL submission

Abstract

Diffusion language models generate text by iteratively denoising all tokens in parallel, but when and where their hidden states encode whether the output will be functionally correct remains unknown. We present the first probing study of diffusion LLM internals, training linear classifiers on hidden states extracted at intermediate denoising steps to predict functional correctness. Across two models (LLaDA-8B, Dream-7B) and four tasks (JSON schema validation, math reasoning, code generation, science QA), we find that (1) correctness signal is already present at step 0, before any denoising occurs, (2) signal emergence is task-dependent, with structural tasks showing flat profiles and reasoning tasks showing gradual buildup, and (3) layer dynamics diverge across architectures. We further demonstrate that per-step probe confidence can identify instances likely to fail, enabling selective generation that avoids 36–98% of wasted compute depending on the task.

1 Introduction

Diffusion language models (dLLMs) have emerged as a promising alternative to autoregressive (AR) generation (Austin et al., 2021a; Sahoo et al., 2024; Lou et al., 2024). Unlike AR models, which generate tokens sequentially, dLLMs produce all output tokens simultaneously through iterative denoising: starting from a fully masked or noisy sequence and progressively refining it over many steps.

This parallel generation process raises a fundamental question: *when* during denoising, and *where* within the model's layers, does the network encode whether its output will be functionally correct? In AR models, probing classifiers have revealed how linguistic and factual knowledge localizes across layers and positions (Tenney et al., 2019; Geva et al., 2023; Azaria and Mitchell, 2023). However, no analogous study exists for diffusion LLMs, whose bidirectional, iterative generation process creates a fundamentally different representational landscape.

We address this gap with the first probing study of dLLM hidden states. Our contributions are: (1) we show that correctness signal exists from step 0, exposed simultaneously across all generation positions, and that dLLMs uniquely gain additional signal through denoising, a capacity absent in single-pass AR decoding; (2) we identify task-dependent emergence patterns, with structural tasks showing strong signal from step 0 and reasoning/code tasks requiring iterative refinement; and (3) we reveal divergent layer dynamics across architectures, with LLaDA concentrating signal in upper layers while Dream migrates signal from upper to lower layers during denoising.

2 Background

Diffusion language models. LLaDA (Nie et al., 2025) uses block masking during training and generates by progressively unmasking tokens over a fixed schedule. Dream (Ye et al., 2025) employs a global masking schedule with a different unmasking strategy. Both models condition on a fully visible prompt and iteratively denoise a generation region of masked tokens.

Probing classifiers. Probing involves training simple classifiers on frozen model representations to test whether specific information is encoded (Belinkov, 2022; Conneau et al., 2018). We adopt this methodology for dLLMs, probing hidden states at intermediate denoising steps.

Related work. Prior studies have probed AR model representations for linguistic structure (Tenney et al., 2019), factual knowledge (Geva et al., 2023), and truthfulness (Azaria and Mitchell, 2023). Confidence-based methods for AR models use early-exit thresholds to skip computation (Schuster et al., 2022; Elbayad et al., 2020). Two concurrent

works study dLLM representations from complementary angles. Shnaidman et al. (2025) demonstrate activation steering for safety behaviors in masked diffusion LLMs, showing that dLLM representations are amenable to linear intervention. Wang et al. (2025) identify temporal oscillation in dLLM outputs, where correct answers appear mid-denoising but are overwritten in later steps. Mündler et al. (2025) propose constrained decoding with context-free grammars to enforce syntactic correctness in dLLM outputs; our work instead asks whether the model's own representations already encode whether the output will be correct, without external constraints. We complement these prior findings by probing hidden representations for functional correctness across the full layer-step grid, characterizing *when* and *where* correctness signal emerges.

3 Method

Hidden state extraction. Given a dLLM with L layers, we run 128-step denoising and extract hidden states at 7 checkpoint steps $t \in \{0, 1, 4, 16, 32, 64, 127\}$. At each checkpoint, we obtain the full hidden state tensor of shape $(L, \text{gen\_length}, d)$. We partition the generation region into 4 equal-length position regions and mean-pool within each region, yielding features of shape $(L, 4, d)$ per instance per step. We choose 4 regions to align with LLaDA's block length of 32 tokens (gen_length / block_length = 4 for JSON schema's 128-token blocks) while keeping the number of features manageable given the limited sample sizes. An ablation over 1, 2, and 4 regions shows best AUC varies by less than 0.02 across configurations (Appendix G).

Probe training. For each (step, layer, region) triple, we reduce dimensionality with PCA to 64 components, apply standard scaling, and train a logistic regression classifier to predict functional correctness (Pedregosa et al., 2011). We evaluate with 5-fold stratified cross-validation, reporting AUC as our primary metric. Control probes trained on shuffled labels yield AUC ≈ 0.50 across all settings, confirming that the observed signal is genuine and not an artifact of the probing pipeline.

Functional correctness. For JSON schema validation, an output is correct if it parses as valid JSON and matches the reference structure. For math reasoning (GSM8K), an output is correct if the extracted numeric answer matches the ground truth. For code generation (MBPP), an output is correct if the generated function passes all provided test assertions. For science QA (ARC), an output is correct if the extracted answer letter matches the gold label.

	JSON	GSM8K	MBPP	ARC
LLaDA-8B	48.5%	66.3%	40.9%	78.2%
Dream-7B	46.0%	61.5%	44.4%	74.5%

Table 1: Baseline functional correctness rates (seed=0, 128 denoising steps).

Selective generation. We simulate instance-level filtering by training a per-step probe (using the best layer and region from the full analysis). At inference time, if the probe predicts with confidence exceeding a threshold τ that an instance will fail, we skip the remaining denoising steps for that instance entirely, producing no output rather than wasting compute on a generation predicted to be incorrect. We report the fraction of instances filtered and the probe's classification accuracy.

4 Experimental Setup

Models. We evaluate LLaDA-8B-Instruct (33 layers) (Nie et al., 2025) and Dream-7B-Instruct (29 layers) (Ye et al., 2025), two open-weight diffusion LLMs with different masking strategies.

Datasets. We use four tasks that test different capabilities: (1) **JSON schema validation** (272 instances from json-mode-eval-extended), a structural task requiring syntactically and semantically correct JSON output; (2) **GSM8K** (1,319 test instances) (Cobbe et al., 2021), a multi-step math reasoning task; (3) **MBPP** (257 sanitized test instances) (Austin et al., 2021b), a code generation task requiring executable Python functions; and (4) **ARC-Challenge** (1,172 test instances) (Clark et al., 2018), a multiple-choice science reasoning task. Generation lengths are 256, 512, 256, and 256 tokens, respectively.

Baseline functional rates. Table 1 shows the functional correctness rates with 128 denoising steps. All datasets have reasonable class balance, enabling meaningful probe training.

5 Results

5.1 Task-Dependent Emergence

Figure 1 shows AUC heatmaps for JSON schema and GSM8K (MBPP and ARC in Appendix B). Two distinct emergence patterns are visible.

For **JSON schema**, correctness signal is already strong at step 0 (AUC $\approx$ 0.78–0.80 $\pm$ 0.04) and remains relatively flat across all denoising steps. This indicates that the prompt encoding alone, before any denoising occurs, already contains signal predictive of structural correctness.

For **GSM8K**, step-0 signal is already well above chance (AUC $\approx$ 0.67–0.71 $\pm$ 0.03), but denoising brings substantially more gain: AUC rises to $\approx$ 0.78–0.82 $\pm$ 0.03 by step 127 (Figure 2).

MBPP follows the same gradual-buildup pattern as GSM8K: step-0 AUC is $\approx$ 0.75–0.76 $\pm$ 0.04 and peaks at $\approx$ 0.84–0.87 $\pm$ 0.05 at later steps (step 64 for LLaDA, step 127 for Dream).

ARC-Challenge shows the weakest and most denoising-dependent signal: step-0 AUC is $\approx$ 0.61–0.67, and signal builds gradually to $\approx$ 0.71–0.75 by step 64–127. Dream's ARC signal is particularly flat until a sharp jump at the final step (0.61 $\rightarrow$ 0.71), suggesting science reasoning requires nearly complete denoising before correctness is predictable.

All four tasks have above-chance step-0 signal, but JSON schema gains only ΔAUC $\approx$ 0.04 from denoising while GSM8K, MBPP, and ARC gain $\approx$ 0.08–0.11, reflecting the difference between structural and reasoning tasks. Standard deviations across 5-fold CV are $\leq$ 0.05 for all configurations, indicating stable estimates. Because dLLMs process all generation positions jointly via bidirectional attention, correctness-relevant information is available across the full output region from step 0.

We also tested whether the probe could select better random seeds: scoring 5 seeds per instance at step 1 and running full denoising only for the top-scoring seed yields +0.0% improvement for both models on JSON schema, confirming that the step-0/1 signal reflects instance-level difficulty rather than seed-specific trajectory quality.

5.2 Divergent Layer Dynamics

The heatmaps in Figure 1 reveal strikingly different layer dynamics between the two architectures (full per-step best layers in Table 3, Appendix C).

LLaDA concentrates its best probing signal consistently in upper layers (L22–28) across JSON schema, GSM8K, and ARC, similar to patterns observed in AR transformers. MBPP is an exception, where best layers are more variable (L15–31).

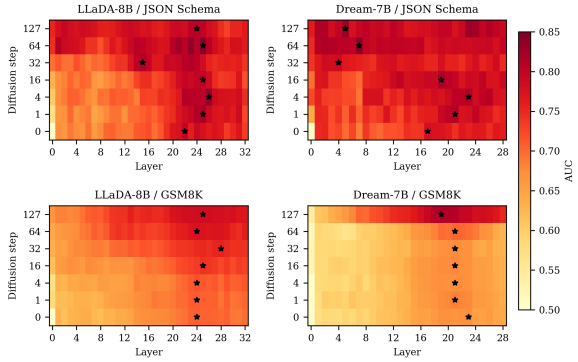

Figure 1: AUC heatmaps (layer $\times$ step) for JSON schema and GSM8K. Each cell shows the best AUC across the 4 position regions for that (layer, step) pair. Stars mark the best layer per step. JSON schema shows strong signal from step 0; GSM8K shows gradual emergence. Note Dream's layer migration on JSON schema. Per-cell standard deviations across 5-fold CV are $\leq$ 0.05. MBPP and ARC heatmaps in Appendix B.

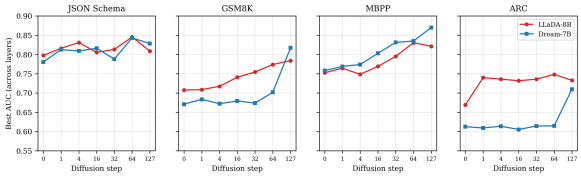

Figure 2: Best AUC across layers at each denoising step. Each point shows the best AUC across all layers and position regions at that step. JSON schema is flat ($\approx$0.80 from step 0), while GSM8K, MBPP, and ARC show gradual emergence. Standard deviations across 5-fold CV are $\leq$ 0.05 for all points.

Dream shows a task-dependent migration pattern. On JSON schema, the best layer migrates from the upper range (L17–23 at steps 0–16) to lower layers (L4–7 at steps 32–127). On GSM8K, MBPP, and ARC, however, Dream's best layers remain in the mid-to-upper range (L15–25) throughout denoising, with no downward migration. This suggests that migration occurs only for tasks where correctness is largely determined early in denoising.

We hypothesize that this difference relates to training origin: LLaDA is trained from scratch, while Dream is fine-tuned from Qwen-2.5 and may inherit its pretrained layer hierarchy, redistributing to lower layers only when the task is simple enough.

	JSON	GSM8K	MBPP	ARC
LLaDA-8B				
Compute saved	98.4%	61.3%	95.5%	82.9%
Clf. accuracy	73.5%	73.5%	66.5%	79.6%
Dream-7B				
Compute saved	96.7%	36.0%	97.8%	61.4%
Clf. accuracy	72.4%	73.5%	70.0%	75.9%

Table 2: Selective generation results ($\tau = 0.80$). Compute saved is the fraction of denoising steps avoided by skipping instances predicted to fail (no output is produced for skipped instances). Classification accuracy (Clf. acc) is the fraction of instances for which the probe correctly predicts functional vs. non-functional.

5.3 Selective Generation

We evaluate whether per-step probe confidence can identify instances whose outputs will be incorrect, enabling the system to skip generation entirely for hopeless instances. Table 2 shows results with a confidence threshold of $\tau = 0.80$.

The results are strongly task-dependent. For JSON schema and MBPP, both models save over 95% of compute: the probe is confident from step 0, identifying most instances predicted to fail immediately. For GSM8K and ARC, savings are more modest (36–83%) because reasoning tasks require more denoising steps before the probe can confidently distinguish hopeless instances. Note that for ARC, where the baseline functional rate exceeds 74% (Table 1), classification accuracy is only marginally above the majority-class baseline, indicating limited discriminative value of the probe on this task.

6 Discussion

Why does step-0 signal exist? At step 0, the dLLM has performed a forward pass over the prompt with masked outputs, producing representations that correlate with eventual correctness. For structural tasks like JSON generation, this correlation is strong: the representation already contains signal predictive of correctness. For reasoning and code tasks, step-0 signal is weaker, and representations are iteratively refined through denoising. The dLLM step-0 signal spans all generation positions simultaneously, arising from bidirectional attention over prompt and masked tokens. An AR baseline probe on Qwen-2.5-7B's last prompt token achieves comparable AUC on most tasks (Appendix F), suggesting that much of the step-0 signal reflects prompt difficulty detectable by any model. On JSON schema, dLLM step-0 AUC modestly exceeds the AR probe (0.78–0.80 vs. 0.74); on other tasks, the gap is negligible (Appendix F, Table 6). The distinctive advantage of dLLMs lies not in step-0 signal strength but in the capacity to gain additional signal through denoising, a capacity absent in single-pass AR decoding entirely.

Relationship to temporal oscillation. Wang et al. (2025) observe that dLLM outputs can oscillate during denoising: correct answers appear at intermediate steps but are overwritten later. Our probe measures whether hidden states *encode* correctness, not whether the current surface output is correct. On JSON schema, hidden-state signal is flat from step 0 while surface tokens are still being refined, suggesting that hidden-state signal predictive of correctness is established early and oscillation reflects surface-level instability rather than changes in the underlying representation.

Layer dynamics and training origin. LLaDA (trained from scratch) shows stable upper-layer localization resembling AR models, while Dream (fine-tuned from Qwen-2.5) exhibits layer migration on simple tasks but not complex ones. This suggests AR pretraining establishes an upper-layer bias that persists for difficult tasks, but diffusion fine-tuning enables redistribution to lower layers when the task is simple enough.

7 Conclusion

We present the first probing study of diffusion LLM internals, showing that correctness signal exists from step 0 (AUC 0.61–0.80), with task-dependent emergence and divergent layer dynamics across architectures. Selective generation based on probe confidence can avoid 36–98% of wasted compute. While an AR baseline achieves comparable step-0 AUC, dLLM probes uniquely gain signal through denoising. Length-stratified controls confirm the signal is not a length confound: AUC drops by 0.00–0.09 after length matching, with MBPP showing no drop at all.

Future work includes validating selective generation in a real inference pipeline, scaling to larger dLLMs, and exploring probe-guided adaptive compute allocation. Nonlinear or multi-layer probes may capture richer structure, and comparing against lightweight heuristics such as per-step token entropy would clarify the added value of learned probes.

Limitations

Our probe is a simple logistic regression classifier with PCA-reduced features; nonlinear probes or multi-layer probes might reveal richer structure. The selective generation analysis is a simulation on states from full 128-step generation and does not account for inference overhead of running probes in real time.

We study only two models (LLaDA-8B, Dream-7B) and four tasks; broader coverage across architectures (e.g., MDLM, SEDD) and model scales is needed. LLaDA and Dream use different unmasking schedules (block-based vs. global linear), so the same step index does not correspond to the same fraction of tokens unmasked across models, complicating direct cross-model temporal comparisons in Figures 1 and 2. A length control probe achieves higher AUC than the correctness probe in most settings (Appendix E); after length matching, correctness AUC drops modestly but remains well above chance. Prompt-length-matched probes similarly show only 0.02 average AUC drop (Appendix I). Other surface-level confounders (formatting tokens) are not controlled for.

Probe selection (best layer and region) uses the same cross-validation splits as evaluation. A nested CV reanalysis (Appendix H) shows AUC drops by only 0.01 on average (max 0.07), confirming that the main patterns are not artifacts of selection bias. We do not compare against simpler confidence heuristics such as token-level entropy, which could serve as a non-probe baseline.

References

Jacob Austin, Daniel D. Johnson, Jonathan Ho, Danny Tarlow, and Rianne van den Berg. 2021a. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, volume 34, pages 17981–17993.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021b. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Amos Azaria and Tom Mitchell. 2023. The internal state of an LLM knows when it's lying. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 967–976.

Yonatan Belinkov. 2022. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Alexis Conneau, German Kruszewski, Guillaume Lample, Loïc Barrault, and Marco Baroni. 2018. What you can cram into a single $&!#* vector: Probing sentence embeddings for linguistic properties. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2126–2136.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. 2020. Depth-adaptive transformer. In *International Conference on Learning Representations*.

Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. 2023. Dissecting recall of factual associations in auto-regressive language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12216–12235.

Aaron Lou, Chenlin Meng, and Stefano Ermon. 2024. Discrete diffusion modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*.

Niels Mündler, Jasper Dekoninck, and Martin Vechev. 2025. Constrained decoding of diffusion LLMs with context-free grammars. *arXiv preprint arXiv:2502.11727*.

Shen Nie, Fengqi Zhu, Chao You, Xiaojun Zhang, and Zhenguo Ou. 2025. LLaDA: Large language diffusion with mAsking. *arXiv preprint arXiv:2502.09992*.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. 2024. Simple and effective masked diffusion language models. *arXiv preprint arXiv:2406.07524*.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q Tran, Yi Tay, and Donald Metzler. 2022. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems*, volume 35, pages 17456–17472.

Andrey Shnaidman, Philip Marek, Eyal Karpas, and Jonathan Herzig. 2025. Activation steering in discrete diffusion language models. *arXiv preprint arXiv:2505.20389*.

Ian Tenney, Dipanjan Das, and Eleni Pavlick. 2019. BERT rediscovers the classical NLP pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601.

Ziming Wang, Jiahao Luo, Haowei Tang, and Tian Gao. 2025. Time is a feature: Temporal oscillation in discrete diffusion language models. *arXiv preprint arXiv:2505.11229*.

Jiacheng Ye, Jiahui Gong, Yanru Lin, Ting Hin Zhuo, Tong Chen, Xin Jiang, Zhenguo Li, Qun Liu, and Lingpeng Liu. 2025. Dream: Diffusion reasoning with enhanced accuracy and mastery. *arXiv preprint arXiv:2504.16915*.

A Generation and Probing Details

Denoising procedure. Both models run 128 denoising steps. LLaDA uses block-based unmasking: the generation region is divided into blocks of 32 tokens, and each block is denoised sequentially over $128/(\text{gen\_length}/32)$ steps per block. Within each block, the model selects the most confident tokens to unmask at each step. Dream uses a global linear schedule: at step i, the fraction of tokens to unmask is determined by a linearly decreasing timestep schedule from $t = 1$ to $t = \epsilon$, and the most confident masked tokens are unmasked globally.

Prompt construction. For JSON schema, we prepend a system prompt containing the target schema and append a JSON code-fence marker as a generation prefix. For GSM8K, the system prompt instructs the model to solve step-by-step and end with #### followed by the numeric answer. Both models use their respective chat templates.

Hidden state extraction. At each of the 7 checkpoint steps $\{0, 1, 4, 16, 32, 64, 127\}$, we run a forward pass with `output_hidden_states=True` and extract the full hidden state tensor across all layers. The generation region is partitioned into 4 equal-length position regions, and we mean-pool within each region, producing a feature vector of dimension d (4096 for LLaDA, 3584 for Dream) per layer per region per instance.

Probe pipeline. For each (step, layer, region) configuration, we construct the feature matrix $X \in \mathbb{R}^{n \times d}$ and apply: (1) `StandardScaler` to zero-mean and unit-variance normalize features, (2) PCA reduction to 64 components, (3) logistic regression with L2 regularization ($C = 1.0$, LBFGS solver, max 1000 iterations). We use 5-fold stratified cross-validation (random state 42) and report the mean AUC across folds.

Computational resources. All generation and feature extraction use NVIDIA A100 (80GB) GPUs on Modal, with 8 parallel A100s per experiment. Models are loaded in bfloat16 precision. Probe training runs on CPU. Total compute: approximately 30 A100-hours across all experiments.

B Additional Heatmaps

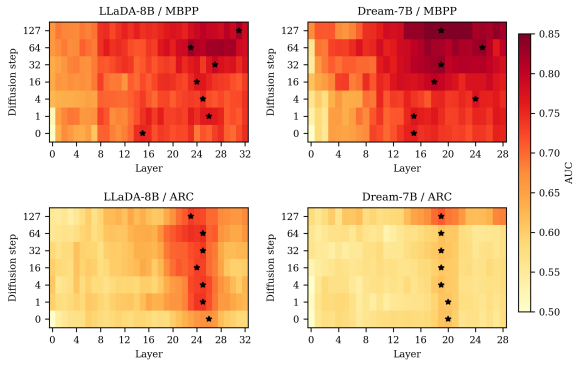

Figure 3: AUC heatmaps for MBPP and ARC-Challenge. Both show gradual emergence similar to GSM8K. ARC has the weakest overall signal (AUC ≤ 0.75).

C Best Layer per Step

Table 3: Best layer (by AUC) at each denoising step. Bold values highlight Dream's migration to lower layers on JSON schema. Other tasks show no such migration.

Step	0	1	4	16	32	64	127
JSON Schema							
LLaDA	22	25	26	25	15	25	24
Dream	17	21	23	19	**4**	**7**	**5**
GSM8K							
LLaDA	24	24	24	25	28	24	25
Dream	23	21	21	21	21	21	19
MBPP							
LLaDA	15	26	25	24	27	23	31
Dream	15	15	24	18	19	25	19
ARC-Challenge							
LLaDA	26	25	25	24	25	25	23
Dream	20	20	19	19	19	19	19

D Selective Generation Details

The selective generation simulation works as follows. For each instance, we iterate through the

checkpoint steps in order. At each step, we compute the probe's confidence as $\max(p, 1-p)$ where p is the predicted probability of functional correctness. If the probe confidently predicts failure (confidence exceeds threshold τ and $p < 0.5$), we skip the remaining denoising steps for that instance, producing no output. If no checkpoint triggers skipping, the instance proceeds to full generation. Compute savings are computed as $1 - \frac{t_{\text{skip}}+1}{128}$, where t_{skip} is the step at which filtering occurs.

The main paper reports results at $\tau = 0.80$. Table 4 shows results across a range of thresholds for all configurations.

Table 4: Selective generation results across thresholds (%). "Sv" is compute saved by skipping instances predicted to fail. "Ac" is classification accuracy.

τ	JSON		GSM8K		MBPP		ARC	
	Sv	Ac	Sv	Ac	Sv	Ac	Sv	Ac
LLaDA-8B								
0.60	99.2	64.0	83.4	67.2	99.1	63.4	98.6	78.1
0.70	99.0	69.5	74.2	70.3	98.2	65.8	94.1	79.4
0.80	98.4	73.5	61.3	73.5	95.5	66.5	82.9	79.6
0.90	93.3	75.0	34.1	74.2	90.8	68.1	57.0	79.9
Dream-7B								
0.60	98.8	63.2	85.3	56.9	99.1	72.0	97.2	74.8
0.70	97.9	69.1	66.3	62.5	98.8	70.0	86.1	75.7
0.80	96.7	72.4	36.0	73.5	97.8	70.0	61.4	75.9
0.90	87.3	74.6	6.2	74.8	94.9	73.5	24.2	76.8

E Length Control Probe

We train a control probe (same pipeline) to predict binary output length (above/below median) instead of correctness. To control for the confound, we re-train the correctness probe on a length-matched subsample where functional and non-functional instances have identical length distributions (binned matching, 10 quantile bins).

Table 5: Best AUC for correctness, length control, and length-matched correctness probes. After controlling for length, correctness AUC drops modestly but remains well above chance.

	Corr.	Length	Corr. (matched)
JSON Schema			
LLaDA	0.85	0.98	0.81
Dream	0.84	0.99	0.75
GSM8K			
LLaDA	0.79	0.86	0.75
Dream	0.82	0.87	0.80
MBPP			
LLaDA	0.84	0.78	0.84
Dream	0.87	0.80	0.87
ARC			
LLaDA	0.75	0.96	0.75
Dream	0.71	0.98	0.72

F AR Baseline Probe

We probe Qwen-2.5-7B-Instruct (Dream's AR base model) using the last prompt token hidden state. Sub-A predicts dLLM labels; Sub-B uses Qwen's own greedy outputs.

Table 6: AR vs. dLLM step-0 probe AUC.

	JSON	GSM8K	MBPP	ARC
Qwen Sub-A (dLLM)	0.74	0.69	0.76	0.67
Qwen Sub-B (AR)	0.77	0.77	0.69	0.74
LLaDA step-0	0.78	0.71	0.76	0.67
Dream step-0	0.80	0.71	0.76	0.61

G Spatial Pooling Ablation

We pool the 4 extracted regions into 1 (global average), 2 (pairwise average), or 4 (concatenation) groups and re-run the probe.

Table 7: Best AUC by number of position regions. Spread is less than 0.03 within each configuration.

	1 region	2 regions	4 regions
JSON Schema			
LLaDA	0.848	0.854	0.840
Dream	0.846	0.828	0.847
GSM8K			
LLaDA	0.784	0.788	0.784
Dream	0.818	0.814	0.812
MBPP			
LLaDA	0.842	0.860	0.834
Dream	0.864	0.868	0.856
ARC-Challenge			
LLaDA	0.747	0.740	0.744
Dream	0.710	0.711	0.716

H Nested Cross-Validation

To verify that probe selection does not inflate reported AUC, we rerun all experiments with nested cross-validation: an inner 3-fold CV on training data selects the best (layer, region), and the outer 5-fold evaluates on held-out data.

Table 8: AUC drop from nested vs. standard CV. Average drop is 0.01; all qualitative patterns are preserved.

	Avg Drop	Max Drop
JSON Schema		
LLaDA	0.005	0.050
Dream	0.035	0.072
GSM8K		
LLaDA	0.009	0.019
Dream	0.008	0.014
MBPP		
LLaDA	0.025	0.053
Dream	0.021	0.038
ARC-Challenge		
LLaDA	0.002	0.015
Dream	0.005	0.031

I Prompt Length Control

We re-train correctness probes on subsamples where functional and non-functional instances have matched prompt length distributions (binned matching, 10 quantile bins).

Table 9: AUC after controlling for prompt length. Average drop is 0.02.

	Original	Prompt-matched	Drop
JSON Schema			
LLaDA	0.80	0.75	0.052
Dream	0.82	0.80	0.022
GSM8K			
LLaDA	0.78	0.77	0.015
Dream	0.82	0.80	0.017
MBPP			
LLaDA	0.81	0.86	−0.046
Dream	0.88	0.85	0.028
ARC-Challenge			
LLaDA	0.73	0.70	0.030
Dream	0.71	0.68	0.029